用户注册流程

1. **用户端发起请求**：客户端(如postman)填写用户名和密码，向服务器发送post请求，URL指向/auth/register
2. **路由匹配与参数解析**：FastAPI服务端接收到请求，根据URL和HTTP方法匹配到对应的路由参数@app.post("/auth/register")。FastAPI利用Pydantic模型自动解析请求体数据，并进行类型校验
3. **查重**：进入register函数，链接user.db，执行SELECT查询，检查输入的username是否已存在。若存在，抛出HTTPException,状态码 *409 User already exists*，中断流程并关闭数据库
4. **数据写入数据库**：如果username未被占用，执行INSERT语句，将用户信息写入数据库，然后执行conn.commmit()提交事务，随后关闭数据库
5. **返回响应**：后端将成功信息("message": "User created successfully","username": data.username)返回给FastAPI,FastAPI将其序列转化为JSON格式，并携带状态码 *201 Created*返回给客户端

用户数据持久化到数据库而非程序内存的原因

1. **防止数据丢失**：程序内存(比如我下面main.py中的全局字典session)只适合存储短暂的sessionId，不能用来存储用户数据，因为一旦服务器重启或程序崩溃都会导致内存中数据丢失。而SQLite可以将数据持久化存储到硬盘上，重启也能恢复
2. **内存又小又贵**：内存的容量有限且成本高昂，不适合存储大量数据。而数据库依托的硬盘有大量的存储空间
3. **支持高级数据操作**：数据库支持复杂的查询(比如连表，聚合)而单纯的内存存储无法实现